Systems Administration (ASIST)

Practical Theoretical Class 09

Network security – Authentication and privacy.
Public key certificates.
Backup and archive.

Privacy

The network environment, especially in WAN networks, is characterized by great difficulty in controlling access to the data circulating there. The only alternative to ensure privacy is to use encryption algorithms, data access is not hindered, but the data will be unreadable to illegal entities.

Encryption becomes totally irrelevant if not applied in conjunction with authentication procedures. Network authentication procedures are intended to ensure that exchanges of information across the network are being performed with the intended counterparty and not with an intruder (***man-in-the-middle*** attacks). Without this validation, the fact that information is encrypted is no longer a guarantee of privacy.

Authentication - Symmetric-key cryptography

Symmetric-key cryptography

If we assume that only the two legal entities know the secret key, then authentication is guaranteed from the outset.

Secret key distribution can be done in a variety of ways:

- **Manual**: the administrator manually sets the same key on both network nodes.
- **Generated on both nodes**: both nodes use the same algorithm to produce a key from something only both know. Usually in the process of generating the key in both nodes a random element produced by one of them (*challenge* or *salt*) is also included.
- **Using a secure channel**: The secret key is generated by one of the nodes and transmitted to the other through a pre-existing secure channel.
- **Using a Key Distribution Center**: A *Key Distribution Center* (**KDC**) acts as an intermediary for establishing a secret key between two nodes. For this to be possible, both nodes must have a secure channel established with the same KDC.

Public-key cryptography

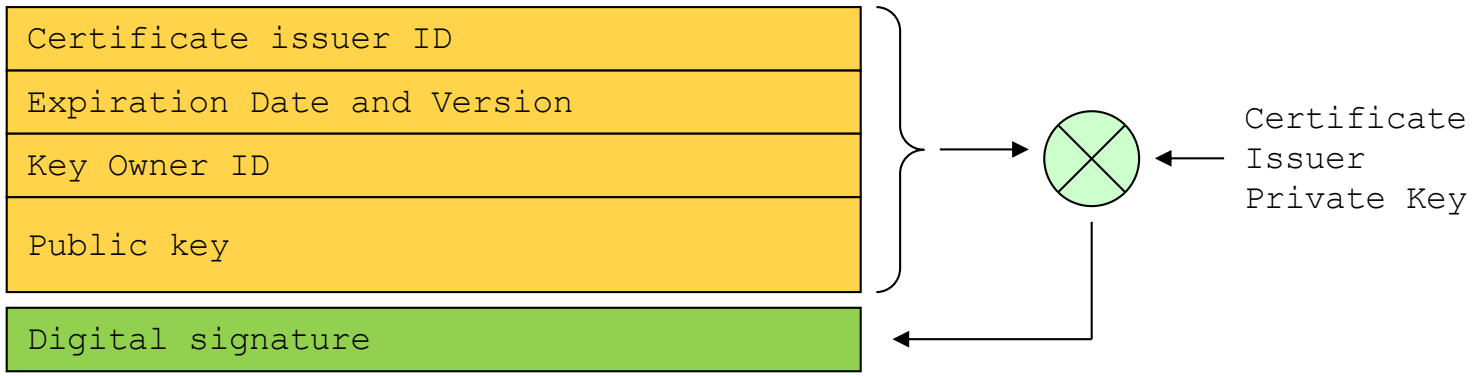
Authentication is not guaranteed because the key used to encrypt is public, so anyone can use it.

However, as the corresponding private key is known only to itself, no one else can decrypt. Given that transactions will occur both ways between the two nodes, if each node is absolutely sure that it is using the counterparty public key, authentication is guaranteed.

Public key certificates

Using a public key algorithm to encrypt a message ensures that only the holder of the corresponding private key can decrypt it, so ensuring the authenticity of the nodes comes down to ensuring the authenticity of the public keys we use.

The authenticity of a public key can be proven through a **public key certificate**. A *public key certificate* is a block of data that contains: the key owner ID (***Subject***) will usually be a DNS name of a network node; the identification of the issuer of the certificate (***Issuer***) that will be a certifying entity, the public key itself and a digital signature of the previous elements produced by the certifying entity:



The certificate will be considered valid if the certifying entity is considered trustworthy, ie if there is trust in the certifying entity. Knowing the public key of the certifying entity can easily prove if the signature was produced by it.

Example:

Certifying entity

Expiration Dates

The owner of the public key you are certifying, typically the server's DNS name

The public key you're making sure

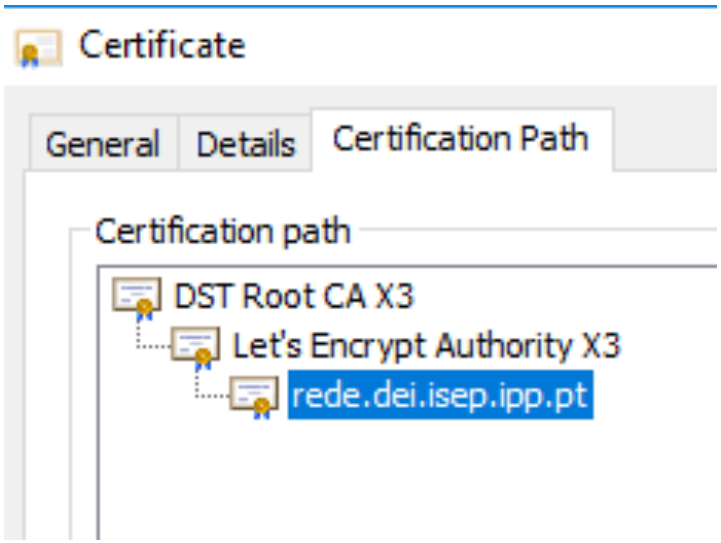
Digital signature of the certifying entity

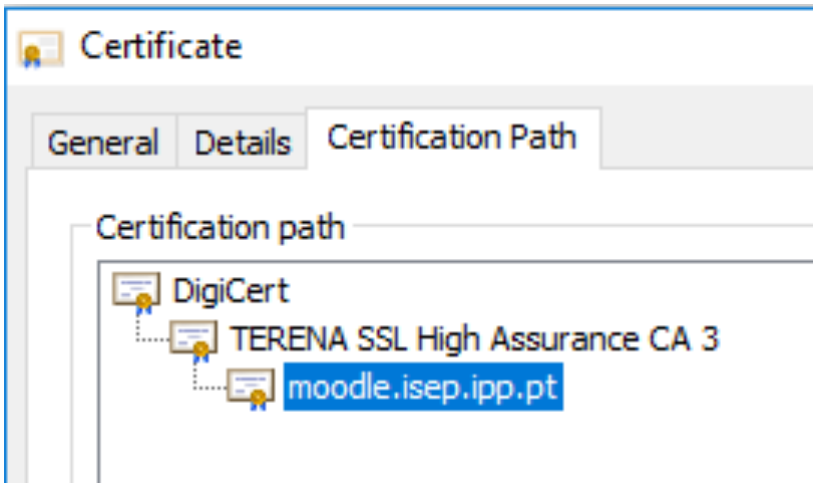
```
[root@server ~]# openssl x509 -in /etc/cert/redecert.pem -text
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number: 2 (0x2)
        Signature Algorithm: md5WithRSAEncryption
        Issuer: C=PT, ST=PORTO, L=PORTO, O=DEI.ISEP, OU=DEI,
    CN=DEICA/emailAddress=dei@dei.isep.ipp.pt
        Validity
            Not Before: Jun 20 11:37:28 2007 GMT
            Not After : Jun 17 11:37:28 2017 GMT
        Subject: C=PT, ST=PORTO, L=PORTO, O=DEI.ISEP, OU=DEI,
    CN=rede.dei.isep.ipp.pt/emailAddress=dei@dei.isep.ipp.pt
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            RSA Public Key: (1024 bit)
                Modulus (1024 bit):
                    00:cc:38:e2:e6:5d:ff:43:6d:ff:43:12:94:03:db:
                    8f:5c:28:61:1a:61:0d:d5:5e:04:dd:20:a9:d8:99:
                    0c:53:e3:c6:23:b0:6c:4d:fe:6b:9e:b8:00:ea:23:
                    1d:55:fa:e7:9a:9b:1b:fa:ef:e0:0d:2c:e0:e1:43:
                    31:2d:b1:37:b5:27:68:01:e0:3d:d8:bf:96:15:bb:
                    25:31:ec:6c:38:5e:2f:17:4c:b5:14:5e:8c:de:1b:
                    14:20:b9:9c:fb:fe:41:5e:ea:68:17:ab:50:a7:9f:
                    6d:93:b3:30:0f:c2:09:2b:7b:43:a4:06:1b:2a:e8:
                    d2:e5:ca:ff:71:e1:69:9c:fb
                Exponent: 65537 (0x10001)
    Signature Algorithm: md5WithRSAEncryption
        97:e9:b2:9d:f1:ca:16:37:43:21:3a:11:61:03:d7:4b:de:f9:
        06:f9:ee:02:4e:6e:08:25:bc:e3:98:e8:1d:bf:9f:43:5b:cb:
        6c:20:d0:6e:7c:d2:53:f2:29:3b:f9:6e:aa:c3:e0:ab:f8:8f:
        06:83:95:9f:dc:a8:94:bb:a8:50:67:34:de:64:0a:02:29:f0:
        1b:1d:f1:bc:51:09:37:f9:15:23:14:5d:b6:98:85:86:d4:37:
        ff:58:d0:74:24:2f:0d:da:d9:c4:02:89:7f:e6:6c:98:c3:f0:
        8e:42:77:45:3d:a1:ae:a2:8c:20:83:83:c5:2b:cc:58:7c:d7:
        2f:06:32:c6:fa:95:b3:de:5f:48:76:54:08:7a:df:b1:58:a8:
        8d:d9:74:94:57:c1:57:5b:79:32:f1:71:02:77:bb:6f:2a:9f:
        42:94:ef:37:d1:03:da:db:9f:b6:11:8a:4b:d4:e4:34:48:86:
        a0:e3:ce:9e:a3:f4:97:86:37:e1:b9:d4:af:de:a8:e9:ce:2a:
        70:aa:8f:c5:90:d9:71:48:a8:9b:8b:f0:2e:6c:42:7e:c3:bf:
        b6:8e:80:79:2f:ed:81:cb:3a:01:69:ce:c0:e8:9f:fe:ce:0a:
        bf:08:89:4b:68:d0:d5:9b:78:9a:e6:e5:19:92:6c:93:7f:c5:
        a0:89:f9:45
```

Certification chains

A public key certificate is considered trusted to the extent that the *certifying authority* (**CA**) that issued it is considered trusted.

Trust propagates between CAs, one CA will be trusted if your public key certificate was issued by another trusted CA. This forms a **certification chain**, but there must be a source for this chain, at the source is a **ROOT CA**.





The list of all trusted ROOT CA's (their public key certificates) is managed by the operating system or application and is updated when the operating system or application is updated.

ROOT CA and self-signed certificates

If ROOT CA is the source of certification chains, who signs the public key certificate for a ROOT CA? The answer is, the ROOT CA itself, ie a self-signed certificate.

A self-signed certificate is trusted by local configuration of the operating system or application by being placed in the trusted certificate list. Thus the corresponding entity and all the certification chains originating from it are considered trustworthy.

Validity of certificates

When a certificate is used in a network environment (typically received from a server by a client) several validations are performed:

- If it is within the expiration date.
- If the **CN** field of the certificate *subject* matches the **DNS name of the server**.
- Whether the certificate issuer is in the chain of a trusted CA.

Depending on the type of client application, if one of these checks fails it may be asked to the user if still wants to accept the certificate and eventually even install that certificate as trusted (at his own risk).

Revocation of public key certificates

Although each certificate contains an expiration date, it may in certain circumstances be necessary to prevent its use before its expiration.

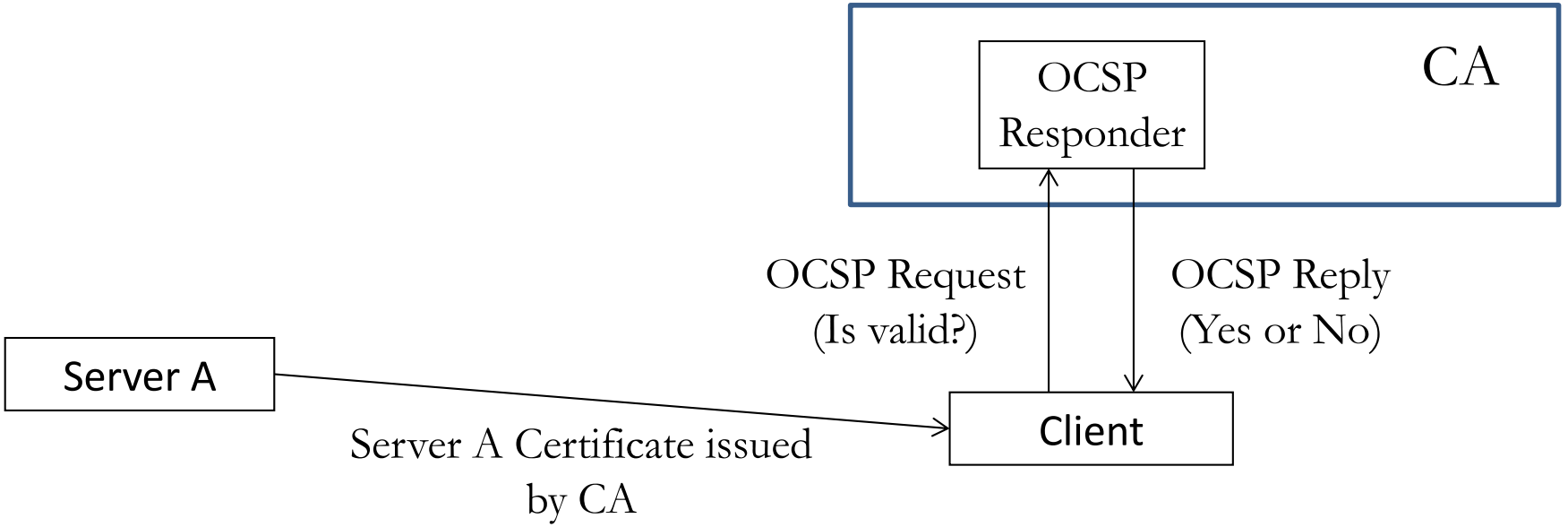
Certificate revocation is required when you become aware that the corresponding private key has been compromised and is owned by illegal entities. If not revoked, users could continue to use it and thus be communicating with an illegal server created by the attackers.

Verification of certificate revocation requires the user to perform additional validation through direct contact with the issuing certification authority.

In order to verify that a certificate has been revoked, the issuer of the certificate must include in it an additional attribute that identifies by means of a DNS name the network service available for that purpose.

OCSP (*Online Certificate Status Protocol*)

While there are other mechanisms, the most widespread way to check if a certificate has been revoked is through **OCSP**. To this end, the CA should provide an OCSP server, commonly referred to as ***OCSP responder***, in addition it must accordingly set an attribute on the certificates it issues where it places the DNS name of its OCSP server.



The level of demand across multiple client applications receiving server certificates varies, some can not accept certificates from a CA if it does not provide a revocation checking service.

Linux – openssl command

The **openssl** command provides a command line interface for openssl library access that supports SSL v2 and v3 and TLS v1. Among other features the openssl command lets you manage public key certificates.

openssl req … – the main purpose is to generate a public/private key pair and issue a public key certification request. The application for certification must then be processed by a certifying entity which will issue the respective certificate. This command can also be used to produce a self-signed certificate, for example:

```
openssl req -nodes -x509 -keyout pri.key -out pub.crt -days 3650 -newkey rsa:1024
```

In this example, the **pri.key** file containing a **1024-bit RSA** private key and the **pub.crt** file containing its self-signed public key certificate will be generated.

openssl x509 … – Certificate processing, for example, is used by the certificate authority to produce the certificate from a certificate request.

Openssl command uses **openssl.cnf** configuration file. Some operations, notably certificate management as a certificate authority, require the use of this configuration file.

Windows Server – Certificate Authority

Windows Server can perform the role of certification authority (CA) in the context of *Active Directory* by enabling the ***Active Directory Certificate Services*** role. You can also install ***Online Responder***.

To operate in the context of *Active Directory*, the CA must be of type ***Enterprise CA***. On a server that is not integrated in an AD domain the CA must be of ***Standalone CA*** type.

The created CA can be inserted into a certificate chain (such as ***Subordinate CA***) using an appropriate certificate issued by another CA, or it can be a ***ROOT CA***, in which case a self-signed certificate will be created.

Through the ***Certification Authority*** application it is then possible to manage the CA, receive certificate requests, issue them and revoke certificates.

Certification Authority Web Enrollment (Web Services), which allows users to request certificates from the CA through the URL **https://SERVER-DNS-NAME/certsrv**, can also be installed.

Backup and archive

There are two main reasons why server administrators can make copies of their server file systems.

Backups – are intended to ensure that in the event of a serious server failure leading to the irretrievable loss of data stored on your file systems, there is at least one copy from which it can be reset. How often these copies are made will determine how far the recovered system will differ from the system state at the time of failure. Another consideration is that, given the objectives, the copy storage location must be physically removed from the original, thus preventing a serious local disaster from affecting both the original and the copy.

Archive – In this case the goal is to free up space on the server file systems, data that is no longer needed is copied to another location and removed from the server. They are still available if needed.

We will only cover existing applications by default on operating systems.

One thing that should be kept in mind when making copies is that they must guarantee the same level of privacy as is guaranteed for the data in the original file systems.

Privacy must be ensured from the point of view of where the copies will be stored and from the point of view of the process of making the copy. In particular if copying over the network should use transfer protocols that include mechanisms that provide authentication and privacy guarantees.

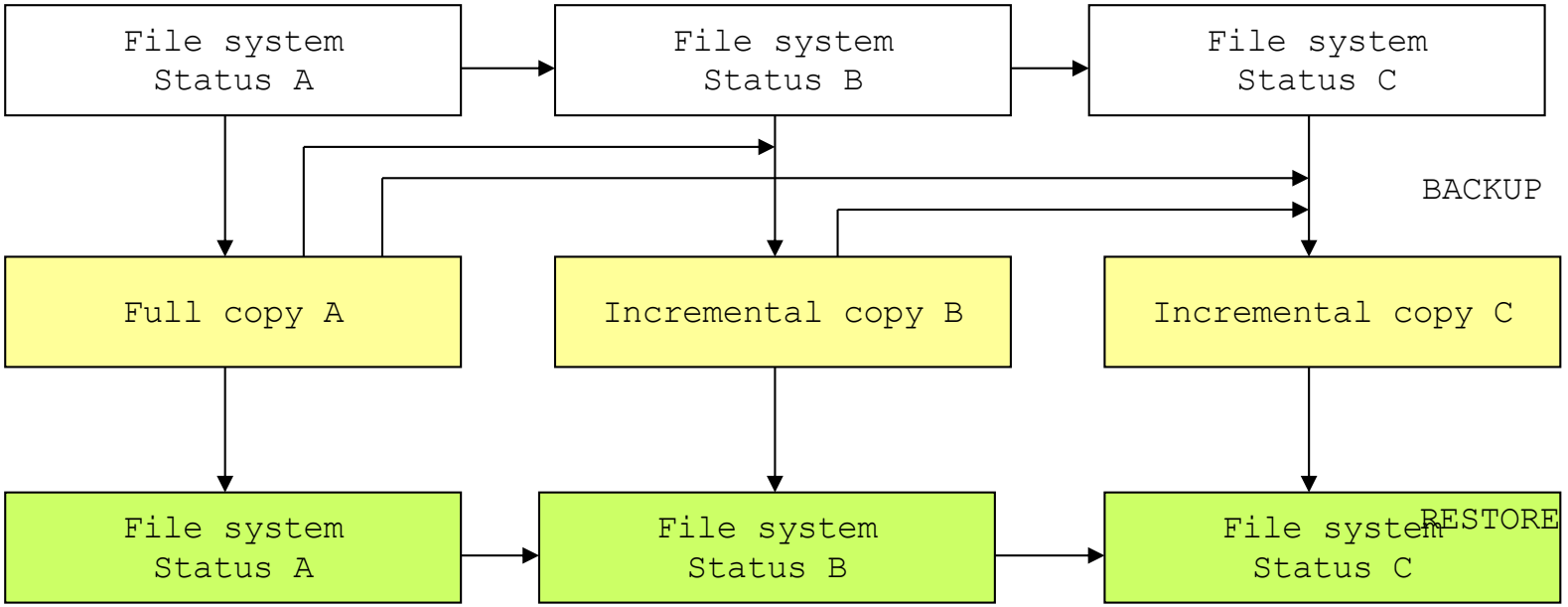
Copies can be made at various levels:

- **Disk** – Full disk copy, including partition table, can usually only be reset to a completely empty identical disk.
- **Partition** – full copy of the partition, replacement requires a partition of the same size as the original.
- **File system** – full file system copy is somewhat similar to the previous one, but is done by copying the file system structures and not the entire partition.
- **File system objects** – may eventually include all objects in a file system, partition or volume. Unlike above it can be done on a file system that is in normal use because objects are copied one by one through the operating system.

Incremental copies

Copying objects through the operating system is often the most used because it is the only one that can be done with the normal working file system, the other types of copying require that the file system be in *read-only* mode as a minimum.

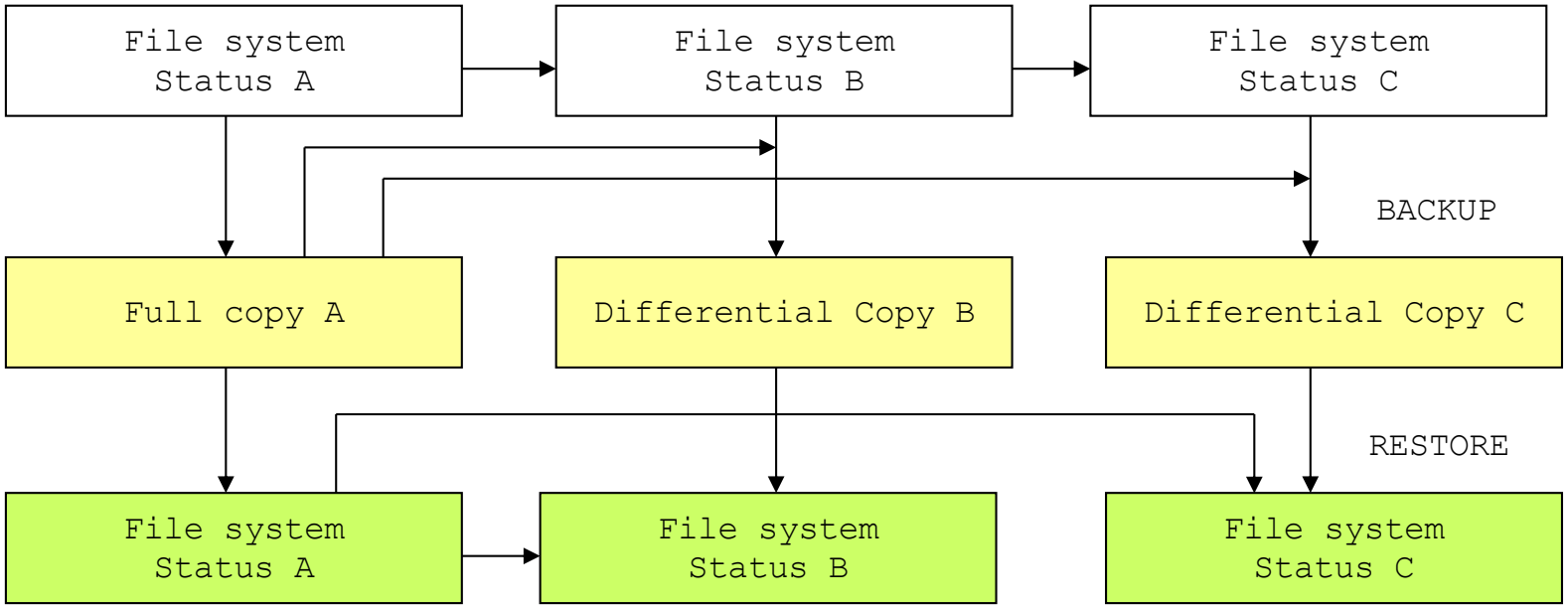
Possible strategies are **full copy** (which can be very time-consuming) followed by incremental copies.



Differential copies

Incremental copies have two drawbacks: both full copy and all subsequent incremental copies must be retained; restore is time consuming because it requires you to apply the full copy and then the multiple incremental copies one by one in the correct order.

Differential copies are slightly different, each differential copy always records system differences from the full copy (not the previous copy).



It is just needed to keep the full copy and the last differential copy, restoring becomes much faster.

Whether incremental or differential, the number of successive process applications should always be limited, the cycle periodically restarted and a full new copy made.

It is a best practice to make a full copy every weekend and to make incremental or differential copies during the week. Full copy, more time consuming, is thus done during a period of less activity.

Backups should be automated through scheduled execution services, with the added benefit that they are performed during periods of low activity.

Copying will always have some impact on server performance and will also be made easier if no significant file system changes are currently taking place.

Windows Server Backup

Windows Server Backup application can be used on Windows Server after feature of the same name is installed.

It allows to realize directly a copy (backup once) or to schedule the execution of copies (backup schedule). You can copy entire drives (volumes), system data, the entire server (bare metal recovery) or just some files and folders.

The copy destination can be either a local drive or a network share, at the destination location a folder will be created such as the name ***WindowsImageBackup*** containing the copies made to that destination and all associated information including the source machine and file system and the date/time it was performed.

Windows Server Backup automatically manages full and differential copies without direct administrator control. Initially makes a full copy and then incremental copies, periodically makes a full copy again.

To restore, the ***recover*** option must be used and select the source (drive or network share containing the ***WindowsImageBackup*** folder), the various copies available in that location will be displayed, all or part of the elements contained in the copy can be restored to original locations or other locations.

Linux – copy commands by default

On Linux there is no ***Windows Server Backup*** equivalent application, at least not with the same degree of operating system integration and simplicity of use.

There are several top-level tools available, many of which are just friendly *front ends* of a set of copy commands that are available on the operating system.

With the help of some scripts and scheduled execution, these commands can be used to implement automatic copying mechanisms for various purposes.

Command tar (*Tape Archive*)

The **tar** command lets you create a file sometimes called ***tarball*** that contains parts or all of a file system.

Once tarball has been created (**-c** *option*), all or part of the files and folders stored in it can be extracted (**-x**) while preserving all original properties (**-p** option).

Additionally tarball can be compressed using gzip (**-z** option) or other compression algorithms. Object names are saved in tarball, but never include the file system root, so objects can be extracted to a different location than the original.

tar supports incremental copying via the **-g** option, this option may indicate a ***snapshot*** log file, if such a file does not exist a full copy will be made and the file will be created, for example:

```
tar -c -f /backups/backup.00.tar -p -g /backups/backup.snap /etc /var/lib
tar -c -f /backups/backup.01.tar -p -g /backups/backup.snap /etc /var/lib
tar -c -f /backups/backup.02.tar -p -g /backups/backup.snap /etc /var/lib
```

Assuming initially that the **/backups/inc.snap** file does not exist, then the **/backups/backup.00.tar** file will be a full copy of the contents of the **/etc** and **/var/lib** folders. The following will be incremental copies. Remember that tar omits the root names of objects, so they will be stored with names beginning with **etc/** and **var/lib/**.

For recovery (extraction) the snapshot log isn't mentioned, but the **-G** option is necessary to indicate that it is a sequence of incremental copies and recovery must be made exactly the same order:

```
tar -x -f /backups/backup.00.tar -p -G -C /
tar -x -f /backups/backup.01.tar -p -G -C /
tar -x -f /backups/backup.02.tar -p -G -C /
```

The –C option indicates where to extract the contents of files, if not specified they will be extracted to the current folder.

Remote copies

Normally copies will be made to remote servers, in case of backups this should be considered as a requirement.

The simplest way to accomplish this is simply to copy to a network share (NAS) with ***read-write*** permission, on Linux it will typically be a remote folder mounted by NFS, but can also be by SMB/CIFS.

An alternative is to use the **rsync** command, this command allows you to copy file systems to remote servers via SSH or through your own service (**rsyncd**) which will need to be installed on the target machine.

Normally the operation will be performed *unattended* so no password can be requested. If *rsyncd* service is used the password can be stored in a file (rsyncd username/password is independent of system users). If SSH is used, it is necessary to configure this service to work without password, the most advisable option is to authenticate through a public key/private key pair.

If the SSH service is configured to operate without passwords it is also possible to send the tar command output through an SSH session, for example:

```
tar -c -p /etc /var/lib | ssh USER@TARGET-HOST "( cat > /backups/backup.tar )"
```

Partial copies

A partial copy covers only part of the system. Usually the goal is to safeguard data related to a given subsystem or service, such as web servers, database servers, etc..

Partial copying can be done on Unix using the **tar** command, for that purpose simply locate all files and folders associated with the subsystem or service concerned.

The following example creates a compressed backup for a particular Apache server installation:

```
tar -c -z -p -f /backups/apache2.tar.gz /etc/apache2 /var/www  \
/etc/ssl/certs/apache2.pem /etc/ssl/private/apache2.key
```

In this case all configuration files in the **/etc/apache2** folder, the pages provided **/var/www** and the keys used by HTTPS are being contemplated.

Many services have their own commands for backing up your data while it is running, especially for databases.

Examples: **MySQL** service has **mysqldump** command, **OpenLDAP** service has **slapcat** command.

In these cases it is clearly advantageous to use these specially designed commands which can be used safely without interruption of service.

Linux – copy systems

While it is possible through scripts and the above commands to automate copying, there is also some higher level free software that should be mentioned.

AMANDA (Advanced Maryland Automatic Network Disk Archiver) (http://www.amanda.org/)

AMANDA is a centralized copy system consisting of a server running on a Linux machine where copies are stored and clients making copies to the server.

Clients may be Linux machines, but there is also a client for Windows systems. Scheduled copying can be triggered from the AMANDA server.

Many of AMANDA's features are implemented through native Linux system commands like tar and ssh.

